AUTOMATED SECURITY ASSESSMENT & REPORTING PIPELINE

รายงานการแก้ไขระบบ ขั้นที่ 4

Gemini ใน pipeline · แก้ bug ข้อมูล CWE · เก็บกวาดฐานข้อมูล · EPSS

ต่อจาก "รายงานการแก้ไขระบบ ขั้นที่ 3"

> โปรเจกต์: `C:\Allprojects\Projects`
> GitHub: `github.com/jjjecop/security-pipe-ine` (private)
> ช่วงเวลา: 9–11 ตุลาคม 2026

1. สรุปสั้นๆ

ขั้นที่ 3 ทำให้ระบบ**ไม่พังง่ายและกู้คืนได้** ส่วนขั้นที่ 4 ทำให้**รายงานบอกได้ว่าควรแก้อะไรก่อน** และข้อมูลในรายงานถูกต้องขึ้น คือแก้ bug ที่ทำให้ช่องโหว่บางตัวไม่มีหมวด OWASP และไม่มีคำแนะนำ แล้วเพิ่มคะแนน EPSS
ที่บอกว่าช่องโหว่ไหนกำลังถูกโจมตีจริง

เรื่อง	ก่อน	หลัง
คำแนะนำจาก Gemini	ต้องรัน `generate_remediation.py` แยกเอง และทำทุก CWE ในฐานข้อมูล	`--with-remediation` ทำเฉพาะ CWE ของโปรเจกต์นั้นในคำสั่งเดียว
Semgrep บางกฎ	`cwe_id` ถูกบันทึกเป็น `"C"` ช่องโหว่ 4 จุดไม่มีหมวดและคำแนะนำ	ได้ `CWE-352` / `CWE-319` ถูกต้อง พร้อมหมวดและคำแนะนำ
CWE แปลกๆ	`cwe-79` ถูกทิ้ง และ `0` กลายเป็น `CWE-0`	`CWE-79` และ `None` ตามลำดับ
ลำดับในรายงาน	Top 6 ได้ `risk 9.4` เท่ากันหมด	เรียงตามโอกาสถูกโจมตีจริง (EPSS) ตัวบนสุด 10.0 และมีป้าย EPSS บอกเหตุผล
ฐานข้อมูล	ScanJob 27 ค้าง `running` และมีตาราง backup ที่ไม่อยู่ในโค้ด	ลบแล้ว `alembic check` ผ่าน
ตั้งค่าเครื่องใหม่	ไม่รู้ว่าต้องตั้งตัวแปรอะไรบ้างใน `.env`	มี `.env.example` และ README อธิบาย
จำนวน test	17	33 (และรันอัตโนมัติบน GitHub ทุก push)

ลำดับงานที่ทำ

ข้อ	งาน	หัวข้อในเล่มนี้
1	C1: `--with-remediation`	2
2	แก้ bug `cwe_id = "C"`	3
3	งานเล็ก: README, `.env.example`, test `normalize_cwe_id`, ลบข้อมูลค้าง	4
4	C2: EPSS	5

Commit ในรอบนี้

commit	วันที่	ข้อความ
`fe43fff`	2026-10-09	Add --with-remediation to run Gemini remediation inside the pipeline
`a5b1b4b`	2026-10-09	Fix Semgrep CWE/OWASP parsing when metadata is a string
`e60462e`	2026-10-11	Add .env.example and document --with-remediation in README
`8468e62`	2026-10-11	Accept lowercase CWE IDs and treat CWE 0 as missing
`5acc941`	2026-10-11	Add EPSS to risk score formula
`8c9c6e6`	2026-10-11	Fetch EPSS from FIRST.org and apply it in the pipeline
(รอ commit)	2026-10-11	Show EPSS in report and dashboard

2. C1: ใส่ Gemini เข้าไปใน pipeline

1 เขียน `generate_remediation.py` ใหม่ให้เรียกใช้ได้

ปัญหาเดิม

- โค้ดทั้งหมดรันทันทีตอน import ไม่มีฟังก์ชัน `run_pipeline.py` เลยเรียกใช้ไม่ได้
- สร้าง Gemini client ตั้งแต่ตอน import ถ้าไม่มี API key ก็พังทันที
- ทำทุก CWE ในฐานข้อมูล ไม่ใช่แค่ CWE ของโปรเจกต์ที่เพิ่งสแกน
- path `CheatSheetSeries/...` และ `mitre/1000.csv` เป็น relative ถ้ารันจากโฟลเดอร์อื่นจะหาไม่เจอ
- ต้องเขียน `time.sleep(13)` ตามหลังการเรียก Gemini ทั้ง 3 จุด ถ้าลืมจุดใดจุดหนึ่งจะโดนจำกัดโควตา

แก้อย่างไร

ส่วนใหม่	ทำอะไร
`get_client()`	สร้าง client ตอนจะใช้จริงครั้งแรกเท่านั้น import ไฟล์ได้แม้ไม่มี key
`BASE_DIR` + `os.path.join`	path แบบเต็ม หาไฟล์เจอไม่ว่ารันจากไหน
`time.sleep` ใน `summarize_with_gemini`	รอเองทุกครั้งที่เรียก อยู่ที่เดียว ลืมไม่ได้
`remediate_one()`	หาคำแนะนำทีละแหล่ง: Cheat Sheet → MITRE → MITRE ของ CWE ลูก (ลำดับเหมือนเดิม)
`get_project_cwe_ids()`	ดึงเฉพาะ CWE ของโปรเจกต์ที่ยัง `open`
`generate_remediation(session, cwe_ids)`	กรอง `remediation_workaround IS NULL` ใน SQL ถ้าครบแล้วจบทันที ไม่ต้องโหลด Cheat Sheet 2.5 GB
`main()` + `--project`	ยังรันเดี่ยวได้เหมือนเดิม

ไฟล์: `generate_remediation.py`

2 เพิ่ม `--with-remediation` ใน `run_pipeline.py`

ลำดับขั้นตอนใน pipeline

```
สแกน → จับคู่ CWE กับ OWASP → (EPSS) → Gemini → สร้างรายงาน
```

ต้องอยู่**หลัง**การจับคู่ CWE เพราะต้องรู้ก่อนว่าโปรเจกต์มี CWE อะไร และอยู่**ก่อน**รายงาน เพื่อให้รายงานมีคำแนะนำใหม่

ทำไมห่อด้วย `try` / `except`

ถ้า Gemini ล้ม (โควตาหมด เน็ตหลุด ไม่มี key) ผลสแกนที่ทำมาแล้วต้องไม่เสียเปล่า รายงานยังต้องออก ใช้ `session.rollback()` ล้างสถานะที่ค้างจาก error ก่อนใช้ session ต่อ

ทำไมไม่เปิดเป็นค่าเริ่มต้น

ใช้เวลา 13–40 วินาทีต่อ CWE และเสียโควตา API ถ้าไม่ตั้งใจจะใช้ก็ไม่ควรเรียก

ผลทดสอบ

ScanJob 34 → `completed` → `CWE ที่ยังไม่มีคำแนะนำ: 0` → สร้างรายงานสำเร็จ

ไฟล์: `run_pipeline.py`

3. แก้ bug `cwe_id = "C"`

3 Semgrep บางกฎถูกบันทึก CWE เป็นตัวอักษรเดียว ข้อมูลผิดแบบเงียบ

เจอได้ยังไง

ระหว่างทดสอบ C1 ผลขึ้นว่า `หา mapping ไม่เจอ: 15 รายการ` เลยเข้าไปดูในฐานข้อมูลว่าเป็นตัวไหนบ้าง:

tool	cwe_id	จำนวน	สาเหตุ
zap	`-1`	10	ปกติ: ZAP ใช้ `-1` แทน "ไม่มี CWE"
trivy	ว่าง	11	ปกติ: บาง CVE ไม่ได้ระบุ CWE
trivy	`CWE-674`	2	ปกติ: ไม่อยู่ใน OWASP Top 10
semgrep	`C`	**4**	**bug**

สาเหตุ

`normalizer.py` คาดว่า `metadata.cwe` เป็น list เช่น `["CWE-352: CSRF"]` แล้วหยิบตัวแรกด้วย `[0]` แต่บางกฎ (`django-no-csrf-token`, `using-http-server`) ส่งมาเป็น **string** `"CWE-352: CSRF"` พอใช้ `[0]` กับ string เลยได้ตัวอักษรแรก `"C"` ส่วน `owasp` ก็มีความเสี่ยงแบบเดียวกัน (จะได้ `"A"`)

ทำไม test เดิมไม่จับ

ข้อมูลทดสอบใน `test_normalizer.py` มีแต่แบบ list ไม่เคยลองแบบ string

4 แก้แบบ "test ก่อน แก้ทีหลัง"

ขั้นตอน

```
เขียน test → รัน: 1 failed ('C' == 'CWE-352') → แก้โค้ด → รัน: 18 passed
```

ถ้าแก้ก่อนแล้วค่อยเขียน test จะไม่รู้ว่า test นั้นจับ bug ได้จริงไหม การเห็น test **fail ก่อน**คือหลักฐานว่า test นี้ป้องกัน bug นี้ได้

test ที่เพิ่ม

```python
def test_semgrep_cwe_and_owasp_as_string():
    raw = make_semgrep_raw(10, "app.listen(80)")
    raw["extra"]["metadata"]["cwe"] = "CWE-352: Cross-Site Request Forgery (CSRF)"
    raw["extra"]["metadata"]["owasp"] = "A01:2021 - Broken Access Control"
    result = normalize_semgrep_finding(raw)
    assert result["cwe_id"] == "CWE-352"
    assert result["owasp_category"] == "A01:2021 - Broken Access Control"
```

แก้โค้ด

เพิ่มฟังก์ชันที่แปลงทุกรูปแบบให้เป็น list ก่อน แล้วโค้ดเดิมที่ใช้ `[0]` ก็ถูกเสมอ:

```
def as_list(value) -> list:
    if value is None:
        return []
    if isinstance(value, str):
        return [value]
    return list(value)

cwe_list = as_list(metadata.get("cwe"))
owasp_list = as_list(metadata.get("owasp"))
```

ไฟล์: `normalizer.py`, `tests/test_normalizer.py`

5 แก้ข้อมูลเก่าที่บันทึกผิดไปแล้ว

ปัญหา

แก้โค้ดแล้ว แต่ 4 แถวในฐานข้อมูลยังเป็น `C` เพราะตอนสแกนเจอ finding เดิม `save_findings.py` อัปเดตแค่ตำแหน่ง severity และคะแนน ไม่อัปเดต `cwe_id`

แก้อย่างไร

เพิ่ม `existing.cwe_id = normalized["cwe_id"]` ในส่วนอัปเดต แล้วสแกน Semgrep ใหม่หนึ่งรอบ ข้อมูลก็ถูกแก้เอง

ทำไมไม่อัปเดต `owasp_category` ด้วย

หมวด OWASP ของ Trivy ถูกเติมทีหลังโดย `apply_cwe_mapping` ถ้าเขียนทับด้วยค่าจาก normalizer (ซึ่งเป็น `None`) หมวดที่จับคู่ไว้จะหายหมด

ผล

กฎ	ก่อน	หลัง
django-no-csrf-token (3 จุด)	`C`, ไม่มีหมวด	`CWE-352`, A01:2021 - Broken Access Control
using-http-server (1 จุด)	`C`	`CWE-319`, A02:2021 - Cryptographic Failures

ทั้งสอง CWE มีคำแนะนำในตาราง mapping อยู่แล้ว รายงานจึงแสดงวิธีแก้ได้ทันที

ไฟล์: `save_findings.py`

> **บทเรียน: 1)** test ผ่านหมดไม่ได้แปลว่าไม่มี bug แปลแค่ว่าไม่มี bug ในกรณีที่เราเขียน test ไว้ **2)** สังเกตตัวเลขแปลกๆ ในผลลัพธ์ แล้วเข้าไปดูว่าเป็นตัวไหน **3)** แก้โค้ดแล้วต้องแก้ข้อมูลเก่าด้วย

4. งานเล็ก

6 `.env.example` และ README

ทำไมต้องมี `.env.example`

`.env` มีรหัสผ่านจริงจึง commit ไม่ได้ ถ้าย้ายเครื่องหรือมีคน clone repo ไป
จะไม่รู้ว่าต้องตั้งตัวแปรอะไรบ้าง `.env.example` มี**แค่ชื่อตัวแปร ค่าลับเว้นว่าง** จึง commit ได้ ใช้ด้วย `Copy-Item .env.example .env` แล้วเติมค่า

ทำไม git ไม่ ignore ไฟล์นี้

ใน `.gitignore` เขียน `.env` ซึ่งตรงกับชื่อนั้นพอดีเท่านั้น

README

เปลี่ยนวิธีสร้าง `.env` ให้ใช้ไฟล์ตัวอย่าง และเพิ่มวิธีใช้ `--with-remediation` (ต้อง clone CheatSheetSeries ก่อน)

เช็กความปลอดภัยก่อน push

`Select-String -Path .env.example -Pattern "PASSWORD=|API_KEY="` ต้องเห็นบรรทัดที่จบด้วย `=` เฉยๆ

ไฟล์: `.env.example` (ใหม่), `README.md`

7 test และแก้ `normalize_cwe_id`

ทำไมสำคัญ

ฟังก์ชันนี้แปลง CWE จากทุกเครื่องมือให้เป็นรูปแบบเดียว (`CWE-95` , `352` , `-1`) ก่อนหาในตาราง mapping ถ้าพังช่องโหว่จะไม่มีหมวดและคำแนะนำโดยไม่มี error เลย แบบเดียวกับ bug `C`

วิธีใหม่: ดูพฤติกรรมจริงก่อน

เรียกฟังก์ชันด้วยค่า 9 แบบแล้วดูผล เจอ 2 จุดที่ผิด:

ค่าที่ส่งเข้า	เดิม	ควรเป็น
`cwe-79`	`None` (ถูกทิ้ง)	`CWE-79`
`0`	`CWE-0` (ไม่มีจริง)	`None` (ZAP ใช้แทน "ไม่มี CWE")

เรื่องใหม่: `@pytest.mark.parametrize`

เขียนฟังก์ชัน test เดียว แล้วให้ pytest รันซ้ำทีละค่าในรายการ ได้ 9 test จาก 2 ฟังก์ชัน
ถ้าวันหลังเจอรูปแบบใหม่ก็แค่เพิ่มบรรทัด

```
@pytest.mark.parametrize("raw", ["-1", "0", "", None, "C"])
def test_missing_or_invalid_cwe_returns_none(raw):
    assert normalize_cwe_id(raw) is None
```

แก้โค้ด

```
cleaned = raw_cwe_id.strip().upper()
if cleaned.startswith("CWE-"):
    cleaned = cleaned[len("CWE-"):]
# ZAP ใช้ 0 หรือ -1 แทน "ไม่มี CWE" จึงรับเฉพาะเลขที่มากกว่า 0
if cleaned.isdigit() and int(cleaned) > 0:
    return f"CWE-{int(cleaned)}"
return None
```

ได้ผลพลอยได้ด้วย: เดิมอะไรที่ขึ้นต้นด้วย `CWE-` จะผ่านหมด (เช่น `CWE-abc`) ตอนนี้ต้องเป็นตัวเลขจริงเท่านั้น

ไฟล์: `apply_cwe_mapping.py`, `tests/test_cwe_mapping.py` (ใหม่)

8 ลบ ScanJob 27 และตาราง backup

ขั้นตอนที่ปลอดภัย

1) `pg_dump` ก่อน (`backups\secpipeline_20261011.dump`) **2)** ดูว่ามีอะไรผูกอยู่ **3)** ลบใน transaction ที่มีตัวเช็ก

ทำไมต้องดูสิ่งที่ผูกอยู่

ข้อมูลผูกกันเป็นลำดับ `scanjob → scan_runs → findings` และ `scanjob → reports` ถ้ามี finding ผูกอยู่ ต้องห้ามลบ (เป็นช่องโหว่จริง) ให้เปลี่ยนสถานะเป็น `failed` แทน

เช็ก	ผล
scan_runs	4 ตัว (72–75) ค้าง `running`
findings / reports	0 / 0 → ลบได้

ลำดับการลบ

ต้องลบ `scan_runs` ก่อน `scanjob` เพราะ `scan_runs` อ้างถึง `scanjob` ถ้าสลับลำดับจะเจอ `violates foreign key constraint`

ตาราง backup

เช็กก่อนว่าตารางจริง `cwe_owasp_mapping` มีครบ (53 CWE มีคำแนะนำทั้ง 53) แล้วค่อย `DROP TABLE` ทั้งสองตาราง (Postgres ยอมให้ `DROP TABLE` อยู่ใน transaction และ `ROLLBACK` ได้)

ผล

`deleted scan_runs=4, scanjob=1` · เหลือ 7 ตาราง · `alembic check` → `No new upgrade operations detected.` โครงสร้างฐานข้อมูลตรงกับ `models.py` ทุกจุด

ตาราง: `scanjob`, `scan_runs`, `cwe_owasp_mapping_backup*` (ข้อมูลอย่างเดียว ไม่ได้แก้โค้ด)

5. C2: EPSS

EPSS คืออะไร ทำไมต้องใช้

risk score เดิมคำนวณจาก severity และ CVSS ซึ่งบอกว่า "**ถ้า**โดนโจมตี จะเสียหายแค่ไหน" แต่ไม่ได้บอกว่า "**มีคนโจมตีอยู่จริงไหม**" รายงานเลยมี Top 6 ที่ได้ 9.4 เท่ากันหมด บอกไม่ได้ว่าควรแก้ตัวไหนก่อน

EPSS (Exploit Prediction Scoring System) จาก FIRST.org อัปเดตทุกวัน บอก**โอกาสที่ CVE จะถูกโจมตีจริงภายใน 30 วัน** มีค่า 0–1

CVE	CVSS	EPSS	ความหมาย
ตัว A	9.8	0.002 (0.2%)	ร้ายแรงในทางทฤษฎี แต่แทบไม่มีใครโจมตี
ตัว B	7.5	0.94 (94%)	ร้ายแรงน้อยกว่า แต่มีคนโจมตีอยู่จริง → **แก้ก่อน**

> EPSS มีเฉพาะช่องโหว่ที่มีเลข **CVE** (ผลจาก Trivy) ช่องโหว่ในโค้ดของเราเองจาก Semgrep และ ZAP ไม่มี CVE จึงได้ risk score เท่าเดิม และ ID แบบ `GHSA-` / `NSWG-` ก็ไม่มี EPSS

9 C2.1 สูตร risk score ใหม่

สูตร

```
risk_score = คะแนนเดิม + (3 × EPSS)    แต่ไม่เกิน 10
```

EPSS	บวกเพิ่ม	ตัวอย่าง (high + CVSS 7.0 = 7.0)
ไม่มี	0	7.0 (เท่าเดิม)
0.01	0.03	7.03
0.9	2.7	9.7
critical + CVSS 9.8 + EPSS 0.95	2.85	12.25 → ตัดเหลือ 10.0

ทำไมเลือกสูตรนี้

อธิบายง่าย ช่องโหว่ที่ไม่มี EPSS ได้คะแนนเท่าเดิม test เก่าไม่พัง และตัวที่ถูกโจมตีจริงขึ้นไปอยู่บนสุด ค่า `EPSS_WEIGHT` และ `MAX_RISK_SCORE` เป็นค่าคงที่ ถ้าอยากปรับน้ำหนักก็แก้บรรทัดเดียว

ทำไม `epss_score=None` เป็นค่าเริ่มต้น

โค้ดเดิมที่เรียก `calculate_risk_score(severity, cvss)` แบบ 2 ค่ายังทำงานได้โดยไม่ต้องแก้

test

เขียน 4 test ก่อน → `4 failed` (`TypeError: unexpected keyword argument 'epss_score'`) → แก้ → `31 passed`

ไฟล์: `risk_scoring.py`, `tests/test_risk_scoring.py`

10 C2.2–C2.3 ดึง EPSS จาก FIRST.org

ไฟล์ใหม่ `epss.py`

ฟังก์ชัน	ทำอะไร
`parse_epss_response()`	อ่าน JSON จาก API แปลงเป็น `{"CVE-...": 0.694}`

`fetch_epss_scores()`	กรองเฉพาะ `CVE-` ตัดตัวซ้ำ แล้วส่งทีละ 100 ตัวเพื่อไม่ให้ URL ยาวเกิน
`apply_epss()`	ใส่ `epss_score` ให้ finding ของ Trivy ที่ยัง open แล้วคำนวณ risk score ใหม่
`main()`	รันเดี่ยวได้: `python epss.py --project NodeGoat`

ทำไมไม่ใช้ library `requests`

ใช้ `urllib` ที่มากับ Python ไม่ต้องเพิ่ม dependency และใช้ ca-bundle ของ Norton จาก `.env` ได้

test โดยไม่ต้องต่อ internet

`tests/test_epss.py` test แค่ `parse_epss_response()` ด้วยข้อมูลปลอมที่หน้าตาเหมือน API จริง GitHub Actions จึงรันได้โดยไม่ต้องเรียก FIRST.org

แก้ `save_findings.py`

ตอนสแกนเจอ finding เดิม ให้คำนวณ risk score ด้วย `existing.epss_score` ที่ดึงไว้แล้ว ไม่อย่างนั้นทุกครั้งที่สแกนใหม่คะแนนจะตกกลับไปเหมือนไม่มี EPSS

ต่อเข้า `run_pipeline.py`

ดึง EPSS อัตโนมัติหลังจับคู่ CWE ห่อด้วย `try` / `except` ถ้า API ล่มรายงานก็ยังออก และมี `--no-epss` ไว้ใช้ตอนไม่มี internet

ผล

`ดึง EPSS ได้ 101 CVE | อัปเดต risk score 200 จาก 210 finding` (อีก 10 ตัวเป็น GHSA / NSWG)

ไฟล์: `epss.py` (ใหม่), `tests/test_epss.py` (ใหม่), `save_findings.py`, `run_pipeline.py`

11 C2.4 แสดง EPSS ในรายงาน

ทำไม

ลำดับเปลี่ยนตาม EPSS แล้ว แต่ถ้าคนอ่านมองไม่เห็นค่า EPSS จะงงว่าทำไมตัวนี้ขึ้นมาบนสุด
รายงานที่ดีต้องบอกเหตุผลของลำดับได้

แก้ 3 ไฟล์ตามทางที่ข้อมูลไหล

ไฟล์	เปลี่ยนอะไร
`generate_report.py`	ส่ง `"epss_score"` ไปให้ template
`report_template.html`	คอลัมน์ EPSS ใน PDF (`-` ถ้าไม่มี) และแก้ `<h2>` ซ้อนกัน
`dashboard_template.html`	ป้าย "EPSS 69.4%" บนแถว และกล่องคำอธิบายตอนกดเปิด

```
{{ "%.1f%%"|format(finding.epss_score * 100) if finding.epss_score is not none else "-" }}
```

`%%` คือเครื่องหมาย % จริง เพราะ `%` ตัวเดียวมีความหมายพิเศษใน format

ไฟล์: `generate_report.py`, `templates/report_template.html`, `templates/dashboard_template.html`

ผลกับ NodeGoat

อันดับ	ก่อน (รายงาน 2026-10-05)	หลัง (รายงาน 2026-10-11)
1	set-value · risk 9.4	**y18n · risk 10.0 · EPSS 69.4%**
2	minimist · risk 9.4	zlib · risk 10.0 · EPSS 19.0%

3	fsevents · risk 9.4	minimist · risk 9.5 · EPSS 4.6%
4	mixin-deep · risk 9.4	mixin-deep · risk 9.5 · EPSS 3.5%
5	bson · risk 9.4	set-value · risk 9.5 · EPSS 2.5%
y18n	อันดับ 7 · risk 8.4	อันดับ 1

> **จุดที่น่าสนใจ:** y18n มีความรุนแรงแค่ "สูง" (แถบสีส้ม) ส่วนตัวที่อยู่ข้างล่างเป็น "วิกฤต" (แถบสีแดง) แต่ y18n มีโอกาสถูกโจมตีจริง 69% เทียบกับ 2.5–4.6% ถ้ามีเวลาแก้ได้ตัวเดียว ควรแก้ y18n ก่อน (อัปเกรดเป็น 4.0.1 / 5.0.5 ขึ้นไป) นี่คือสิ่งที่ EPSS เพิ่มให้รายงาน

6. ปัญหาที่เจอระหว่างทาง และบทเรียน

อาการ	สาเหตุ	วิธีแก้ / บทเรียน
เปิด dashboard แล้วไม่เห็น EPSS	เปิดโฟลเดอร์รายงานเก่า (2026-10-05)	ทุกครั้งที่สร้างรายงานจะได้โฟลเดอร์ใหม่ ดูชื่อโฟลเดอร์จากบรรทัด `สร้างรายงานสำเร็จที่ ...` หรือวันที่มุมขวาบนของ dashboard
`Connection refused` / `failed to connect to the docker API`	Docker Desktop ปิดอยู่ (เปิดเครื่องใหม่)	เปิด Docker Desktop แล้ว `docker compose up -d` ก่อนเริ่มงาน
ข้อความเป็นภาษาอังกฤษแปลกๆ เช่น `mevtwi9jv`	พิมพ์ภาษาไทยตอนแป้นยังเป็นภาษาอังกฤษ	เช็กภาษาของแป้นพิมพ์ (`Win` + `Space` หรือ `Alt` + `Shift`)
คำสั่งติดกันตอน copy	copy หลายกล่องพร้อมกัน	copy ทีละกล่องด้วยปุ่มที่มุมกล่อง
`nothing to commit, working tree clean`	commit ไปแล้วก่อนหน้า	ไม่ใช่ error ดูด้วย `git log --oneline -3`
`หา mapping ไม่เจอ: 15`	มีทั้งกรณีปกติและ bug ปนกัน	อย่ามองข้ามตัวเลข เข้าไปดูว่าเป็นตัวไหนบ้าง (เจอ bug `C` จากตรงนี้)
ScanJob ค้าง `running` หลายตัว	รอบทดลองจากโค้ดเก่าที่ไม่บันทึกสถานะตอนจบ	ดูสิ่งที่ผูกอยู่ก่อนลบเสมอ (findings, reports)

> **รูปแบบการแก้ bug ที่ใช้ในรอบนี้** **1)** สังเกตความผิดปกติจากข้อมูลจริง **2)** ลองเรียกฟังก์ชันดูพฤติกรรมจริง **3)** เขียน test ตามพฤติกรรมที่ถูกต้อง → เห็น fail **4)** แก้โค้ด → เห็น pass **5)** แก้ข้อมูลเก่าที่ผิดไปแล้ว **6)** commit โค้ดกับ test ไว้ด้วยกัน

คำสั่งใหม่ที่ได้ใช้

คำสั่ง	ใช้เมื่อ
`python run_pipeline.py ... --with-remediation`	สแกนแล้วให้ Gemini เขียนคำแนะนำ CWE ใหม่
`python run_pipeline.py ... --no-epss`	สแกนตอนไม่มี internet
`python generate_remediation.py --project X`	สร้างคำแนะนำอย่างเดียว
`python epss.py --project X`	อัปเดต EPSS อย่างเดียว (EPSS เปลี่ยนทุกวัน)
`python -c "from mod import f; print(f('x'))"`	ลองเรียกฟังก์ชันดูผลเร็วๆ
`pytest -v tests\test_xxx.py`	รัน test ไฟล์เดียว แสดงผลทีละกรณี
`git diff <ไฟล์>`	ดูว่าแก้อะไรไปก่อน commit
`alembic check`	เช็กว่าฐานข้อมูลตรงกับ `models.py`
`Select-String -Path <ไฟล์> -Pattern "..."`	ค้นข้อความในไฟล์ (เช่นเช็กว่าไม่มีความลับหลุด)
`Win` + `PrtSc`	แคปหน้าจอ บันทึกไว้ที่ `OneDrive\Pictures\Screenshots`

7. สถานะงาน และสิ่งที่ยังเหลือ

เทียบกับ "งานที่ยังเหลือ" ในรายงานชั้นที่ 3

งาน	สถานะ
C1 `--with-remediation`	**เสร็จ** (ข้อ 1–2)
C2 EPSS ใน risk score	**เสร็จ** (ข้อ 9–11) เหลือ commit ส่วนรายงาน
ScanJob 27 ค้าง `running`	**เสร็จ** (ข้อ 8)
ตาราง `cwe_owasp_mapping_backup*`	**เสร็จ** (ข้อ 8)
`.env.example`	**เสร็จ** (ข้อ 6)
test `normalize_cwe_id`	**เสร็จ** (ข้อ 7)
C3 FastAPI และ Redis worker	**ยังไม่ทำ**
backup อัตโนมัติ	**ยังไม่ทำ**
test `extract_location_detail`, `save_findings`	**ยังไม่ทำ**
สแกนหลาย image ในโปรเจกต์เดียว	**ยังเป็นข้อจำกัด**

เรื่องที่เจอใหม่ (ยังไม่ได้แก้)

เรื่อง	รายละเอียด
ScanJob 18–26 ค้าง `running`	9 ตัว แบบเดียวกับ ScanJob 27: ไม่มี `created_at` ไม่มี finding หรือ report ผูกอยู่ ลบได้ด้วยวิธีเดียวกัน (รอตัดสินใจ)
EPSS เปลี่ยนทุกวัน	ค่าในฐานข้อมูลอัปเดตเฉพาะตอนรัน pipeline หรือ `epss.py` ถ้าไม่ได้สแกนนาน ควรรัน `python epss.py --project X` ก่อนสร้างรายงาน
`datetime.utcnow()`	VS Code เตือนว่าเลิกใช้แล้ว (deprecated) ใน `save_findings.py` ยังทำงานได้ แต่ควรเปลี่ยนเป็น `datetime.now(timezone.utc)` ในอนาคต

ไฟล์ที่เปลี่ยนในรอบนี้

ไฟล์	สถานะ
`generate_remediation.py`	**เขียนใหม่** เป็นฟังก์ชัน, client แบบ lazy, path เต็ม
`run_pipeline.py`	**แก้** `--with-remediation`, EPSS, `--no-epss`
`normalizer.py`	**แก้** `as_list()` สำหรับ cwe/owasp
`save_findings.py`	**แก้** อัปเดต `cwe_id` และใช้ EPSS เดิมตอนสแกนซ้ำ
`apply_cwe_mapping.py`	**แก้** `normalize_cwe_id`
`risk_scoring.py`	**แก้** สูตร EPSS
`epss.py`	**ใหม่** ดึง EPSS จาก FIRST.org
`generate_report.py`, `templates/*.html`	**แก้** แสดง EPSS
`tests/test_cwe_mapping.py`, `tests/test_epss.py`	**ใหม่**
`tests/test_normalizer.py`, `tests/test_risk_scoring.py`	**เพิ่ม test** (รวมทั้งหมด 33)
`.env.example`, `README.md`	**ใหม่** / **แก้**
ฐานข้อมูล	**ลบ** ScanJob 27 + scan_runs 4 แถว + ตาราง backup 2 ตาราง · **แก้** `cwe_id` 4 แถว · **เติม** `epss_score` 200 แถว